

MICSprev workflow

MICSprev authors

Overview

MICSprev provides a workflow for processing MICS survey and GPS data, constructing geographic inputs, fitting direct and model-based estimators, and visualizing results for small area estimation.

This vignette demonstrates a complete example using Nigeria MICS 2021 data. The example covers three indicators:

- neonatal mortality rate (NMR)
- antenatal care coverage (ANC)
- DTP3 vaccination coverage (DTP3)

The main steps are:

1. build geographic inputs from the GPS file
2. read the survey files
3. process each indicator into a standardized format
4. run direct and model-based estimators
5. extract results
6. generate maps and ridge plots

Installation

Required dependencies

This package relies on several external packages that must be installed separately.

Install core dependencies

```
install.packages(c(
  "sf",
  "dplyr",
  "ggplot2",
  "haven",
  "patchwork",
  "stringr",
  "lwgeom"
))
```

Install SUMMER and surveyPrev from GitHub

```
install.packages("remotes")

remotes::install_github("richardli/SUMMER")
remotes::install_github("richardli/surveyPrev")
```

Install INLA (required for model-based estimation)

```
install.packages(  
  "INLA",  
  repos = c(  
    getOption("repos"),  
    INLA = "https://inla.r-inla-download.org/R/stable"  
  ),  
  dep = TRUE  
)
```

Install the development version from GitHub:

```
install.packages("devtools")  
devtools::install_github("SZ-yang/MICSprev")
```

Load the package and supporting libraries:

```
library(MICSprev)  
library(haven)  
library(dplyr)
```

To see the exported functions:

```
ls("package:MICSprev")  
#> [1] "plot_indicator_maps" "plot_indicator_ridge" "process_ANC" "process_DTP3"  
#> [9] "standardize_columns"  
?process_geo_mics
```

Data access

This vignette uses both MICS survey microdata and the corresponding GPS data. These files are not distributed with the package, so users must obtain them directly from the MICS website before running the workflow.

Create an account and log in

First, go to the MICS website:

<https://mics.unicef.org/>

Create an account and log in. Access to both the survey microdata and the GPS data is managed through the MICS website.

Download the survey data

After logging in:

1. Navigate to the **SURVEYS** section.
2. Choose the country and survey year of interest.
For example, for this vignette you would select **Nigeria / MICS6 / 2021**.
3. In the **MICS Datasets** column, click **Available** to download the survey datasets.
4. Download and unzip the files.

For the indicators currently implemented in this workflow, the required files are:

- **bh.sav** for the birth history data

- `wm.sav` for the women's questionnaire data
- `ch.sav` for the children's questionnaire data

Make sure these files all come from the same country and survey year.

Download the GPS data

The GPS data are requested separately from the survey microdata.

1. In the same survey row, go to the GIS column and click Available.
2. Select Request GPS Datasets.
3. Submit the request and wait for approval.
4. After approval, go to your profile and open My GPS Dataset Requests.
5. Download the approved GPS dataset and unzip the file.

After unzipping, you should keep the entire GPS folder. For example, for the Nigeria 2021 survey, the folder may look like:

```
NigeriaMICS2021GPS/
  NigeriaMICS2021GPS.shp
  NigeriaMICS2021GPS.shx
  NigeriaMICS2021GPS.dbf
  NigeriaMICS2021GPS.prj
  NigeriaMICS2021GPS.cpg
  ...
```

Set file paths

```
# Set paths to the unzipped survey data folder and GPS folder
data_dir <- "path/to/unzipped/MICS_survey_data"
gps_dir  <- "path/to/unzipped/NigeriaMICS2021GPS"

# Survey files
bh_path <- file.path(data_dir, "bh.sav")
wm_path <- file.path(data_dir, "wm.sav")
ch_path <- file.path(data_dir, "ch.sav")

# GPS shapefile
gps_path <- file.path(gps_dir, "NigeriaMICS2021GPS.shp")
```

Workflow

1. Build geographic inputs

The first step is to build the cluster- and admin-level geographic objects from the GPS shapefile.

```
# Build geographic inputs from the GPS shapefile
geo <- process_geo_mics(
  country.name = "Nigeria",
  year = 2021,
  gps_path = gps_path
)
```

The returned object is a list that contains geographic information used by downstream estimation and plotting functions. Typical components include:

- `geo$cluster.info`

- `geo$admin.info1`
- `geo$admin.info2`
- `geo$admin1`
- `geo$admin2`
- `geo$geo`

You can inspect its structure with:

```
str(gio, max.level = 1)
#> List of 7
#> $ cluster.info:List of 7
#> $ admin.info1 :List of 4
#> $ admin.info2 :List of 4
#> $ admin0      :Classes 'sf' and 'data.frame': 1 obs. of 7 variables:
#> ..- attr(*, "sf_column")= chr "geometry"
#> ..- attr(*, "agr")= Factor w/ 3 levels "constant","aggregate",...: NA NA NA NA NA NA
#> ..- attr(*, "names")= chr [1:6] "shapeName" "shapeISO" "shapeID" "shapeGroup" ...
#> $ admin1      :Classes 'sf' and 'data.frame': 37 obs. of 7 variables:
#> ..- attr(*, "sf_column")= chr "geometry"
#> ..- attr(*, "agr")= Factor w/ 3 levels "constant","aggregate",...: NA NA NA NA NA NA
#> ..- attr(*, "names")= chr [1:6] "shapeName" "shapeISO" "shapeID" "shapeGroup" ...
#> $ admin2      :Classes 'sf' and 'data.frame': 774 obs. of 9 variables:
#> ..- attr(*, "sf_column")= chr "geometry"
#> ..- attr(*, "agr")= Factor w/ 3 levels "constant","aggregate",...: NA NA NA NA NA NA NA NA
#> ..- attr(*, "names")= chr [1:8] "shapeName" "shapeISO" "shapeID" "shapeGroup" ...
#> $ geo         :Classes 'sf' and 'data.frame': 2079 obs. of 23 variables:
#> ..- attr(*, "sf_column")= chr "geometry"
#> ..- attr(*, "agr")= Factor w/ 3 levels "constant","aggregate",...: NA NA NA NA NA NA NA NA NA NA
#> ..- attr(*, "names")= chr [1:22] "ISO" "MICSC" "SVYROUND" "SVYYEARS" ...
```

2. Read MICS survey files

Next, read the survey files needed for each indicator.

- `bh.sav`: birth history file for NMR
- `wm.sav`: women's file for ANC
- `ch.sav`: children's file for DTP3

```
bh <- read_sav(bh_path)
wm <- read_sav(wm_path)
ch <- read_sav(ch_path)
```

3. Process indicators

The package converts each raw survey file into a standardized analysis-ready data set.

```
dat_nmr <- process_indicator(bh, "NMR")
dat_anc <- process_indicator(wm, "ANC")
dat_dtp3 <- process_indicator(ch, "DTP3")
```

Each processed data set is expected to contain the core variables required by the estimation functions:

- `cluster`
- `householdID`
- `weight`
- `strata`
- `value`

A quick set of sanity checks:

```
lapply(list(NMR = dat_nmr, ANC = dat_anc, DTP3 = dat_dtp3), function(x) {
  list(
    cols_ok = all(c("cluster", "householdID", "weight", "strata", "value") %in% names(x)),
    n       = nrow(x),
    value_tab = table(x$value, useNA = "ifany")
  )
})
#> $NMR
#> $NMR$cols_ok
#> [1] TRUE
#>
#> $NMR$n
#> [1] 27920
#>
#> $NMR$value_tab
#>
#>      0      1
#> 27031  889
#>
#>
#> $ANC
#> $ANC$cols_ok
#> [1] TRUE
#>
#> $ANC$n
#> [1] 10008
#>
#> $ANC$value_tab
#>
#>      0      1
#> 4292 5716
#>
#>
#> $DTP3
#> $DTP3$cols_ok
#> [1] TRUE
#>
#> $DTP3$n
#> [1] 5582
#>
#> $DTP3$value_tab
#>
#>      0      1
#> 2476 3106
```

4. Run models

This section shows how to run the main estimation workflow with `run_indicator_models()`.

4.1 Neonatal mortality rate (NMR)

For NMR, this example fits four model types at admin levels 0, 1, and 2:

- direct estimates
- Fay-Herriot model
- cluster model
- stratified cluster model

The Fay-Herriot model can be configured using `fix_var` and `nested`. In this example, we fit the nested fixed-variance Fay-Herriot model with `fix_var = TRUE` and `nested = TRUE`.

```
fit_nmr <- run_indicator_models(
  data = dat_nmr,
  geo = geo,
  admin_levels = c(0, 1, 2),
  models = c("direct", "fh", "cluster", "cluster_strat"),
  model = "bym2",
  ci = 0.95,
  fix_var = TRUE,
  nested = TRUE,
  verbose = TRUE
)
```

4.2 Antenatal care coverage (ANC)

For ANC, this example fits direct estimates and the stratified cluster model at admin levels 1 and 2.

```
fit_anc <- run_indicator_models(
  data = dat_anc,
  geo = geo,
  admin_levels = c(1, 2),
  models = c("direct", "cluster_strat"),
  verbose = TRUE
)
```

4.3 DTP3 coverage

For DTP3, this example computes direct estimates only at admin levels 0, 1, and 2.

```
fit_dtp3 <- run_indicator_models(
  data = dat_dtp3,
  geo = geo,
  admin_levels = c(0, 1, 2),
  models = c("direct"),
  verbose = TRUE
)
```

5. Access results

The `run_indicator_models()` output organizes results by admin level and model type. This section shows several common extraction patterns.

5.1 National direct estimate for NMR

```
nmr_nat <- fit_nmr$fits[["0"]]$direct$res.natl$direct.est
cat("National NMR per 1,000:", round(nmr_nat * 1000, 2), "\n")
#> National NMR per 1,000: 34.09
```

5.2 National estimate for ANC if admin 0 was requested

```
if ("0" %in% names(fit_anc$fits) && !is.null(fit_anc$fits[["0"]]$direct)) {
  anc_nat <- fit_anc$fits[["0"]]$direct$res.admin0$direct.est
  cat("National ANC coverage:", round(anc_nat, 4), "\n")
} else {
  cat("ANC national estimate not available because admin 0 was not requested.\n")
}
#> ANC national estimate not available because admin 0 was not requested.
```

5.3 Admin 1 direct estimates for NMR

```
nmr_adm1_direct <- fit_nmr$fits[["1"]]$direct$res.admin1
head(nmr_adm1_direct)
```

	admin1.name	direct.est	direct.var	direct.logit.est	direct.logit.var	direct.
#> 1	Abia	0.018945984	7.184776e-05	-3.947036	0.20796636	
#> 2	Abuja Federal Capital Territory	0.010345898	1.595172e-05	-4.560765	0.15216131	
#> 3	Adamawa	0.025354212	4.328944e-05	-3.649129	0.07089051	
#> 4	Akwa Ibom	0.030820510	7.482864e-05	-3.448269	0.08386481	
#> 5	Anambra	0.005807667	4.894342e-06	-5.142752	0.14680813	
#> 6	Bauchi	0.045121376	3.370292e-05	-3.052228	0.01815543	

5.4 Admin 2 Fay-Herriot results for NMR

```
nmr_adm2_fh <- fit_nmr$fits[["2"]]$fh$res.admin2
head(nmr_adm2_fh)
```

	admin2.name.full	mean	median	sd	var	lower	upper
#> 1	Abia_Aba North	0.02006251	0.01797817	0.009351275	8.744634e-05	0.008272062	0.04312294 0.520
#> 2	Abia_Aba South	0.03304969	0.02980647	0.016460183	2.709376e-04	0.011343112	0.07306166 0.552
#> 3	Abia_Arochuku	0.02348421	0.02153590	0.010282190	1.057234e-04	0.009282707	0.04738001 0.477
#> 4	Abia_Bende	0.02787446	0.02528209	0.013018985	1.694940e-04	0.009652283	0.05986901 0.514
#> 5	Abia_Ikuruano	0.02473222	0.02311806	0.008572563	7.348884e-05	0.012125545	0.04396356 0.370
#> 6	Abia_Isiala-Ngwa North	0.03860397	0.03509519	0.016919512	2.862699e-04	0.015275379	0.07961501 0.482

Optional: dropped admin 2 areas for standard FH

The low-variance dropping option only applies to the standard Fay-Herriot model, when `fix_var = FALSE` and `nested = FALSE`. Since the main example uses `fix_var = TRUE` and `nested = TRUE`, no admin 2 areas are dropped in that fit.

5.5 Admin 1 stratified cluster model results for ANC

Some outputs may include a `type` column. The code below safely extracts the full estimates.

```
anc_res_admin1 <- fit_anc$fits[["1"]]$cluster_strat$res.admin1

anc_adm1_strat_full <- if ("type" %in% names(anc_res_admin1)) {
  subset(anc_res_admin1, type == "full")
} else {
  anc_res_admin1
}

head(anc_adm1_strat_full)
```

```

#>               admin1.name      mean   median      sd      var    lower    upper
#> 1             Cross River 0.7799337 0.7823592 0.04072739 0.001658720 0.6928421 0.8518860 0.1
#> 2 Abuja Federal Capital Territory 0.7515335 0.7521719 0.03629113 0.001317046 0.6756825 0.8176028 0.1
#> 3                Ogun 0.7447735 0.7485629 0.03906941 0.001526419 0.6627862 0.8155120 0.1
#> 4                Oyo 0.7622180 0.7625526 0.03949349 0.001559735 0.6786633 0.8329821 0.1
#> 5             Sokoto 0.2650650 0.2626747 0.03775073 0.001425117 0.1961036 0.3437322 0.1
#> 6             Zamfara 0.2190911 0.2178215 0.03220664 0.001037267 0.1606948 0.2866550 0.1

```

6. Plotting examples

This section demonstrates how to generate maps and ridge plots for model comparison. The map examples show posterior means and interval widths, where interval width is defined as the upper interval limit minus the lower interval limit. A CV map is shown once for NMR at admin level 1 as an additional diagnostic.

```

out_dir <- tempfile("MICSprev_plots_")
dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)

```

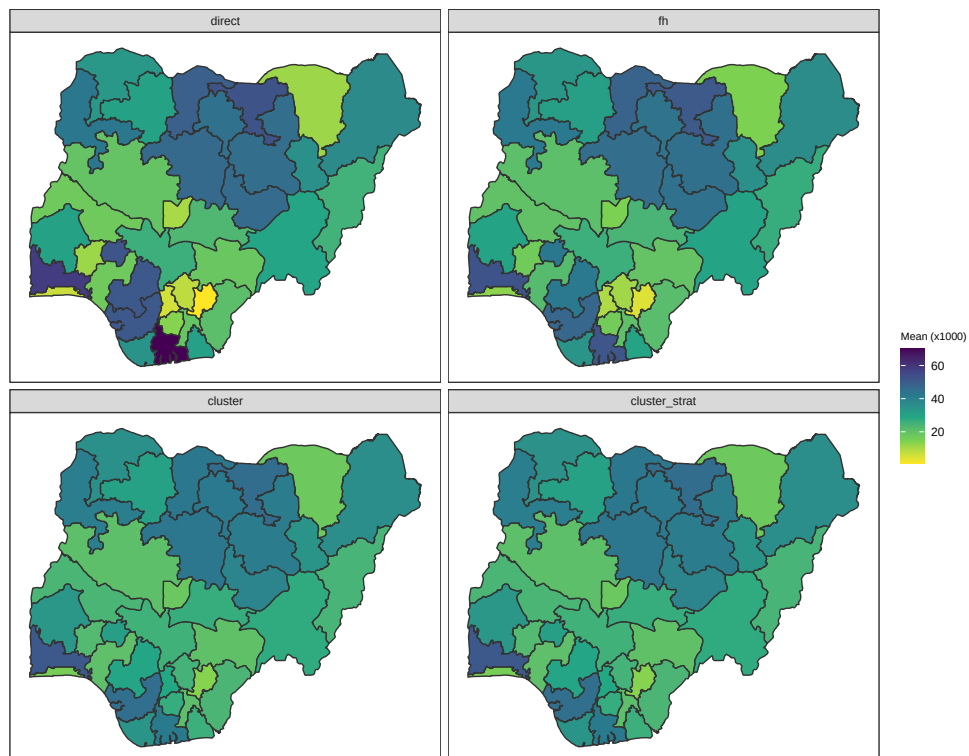
6.1 NMR maps at admin level 1

```

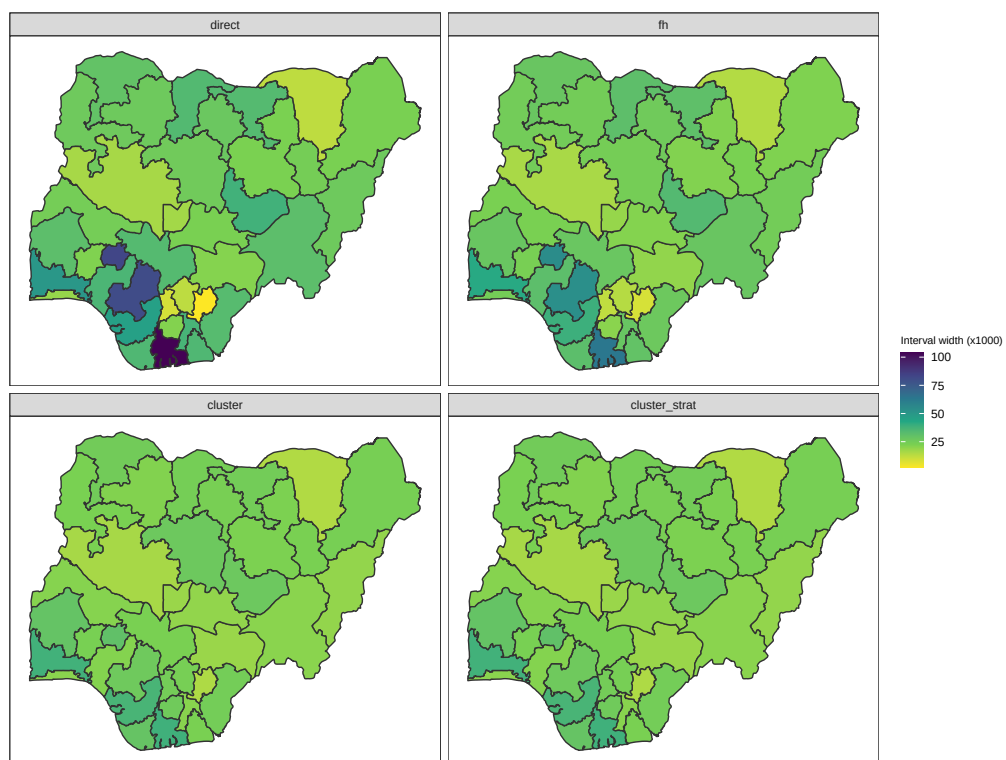
nmr_maps_adm1 <- plot_indicator_maps(
  fit      = fit_nmr,
  geo      = geo,
  admin    = 1,
  models   = c("direct", "fh", "cluster", "cluster_strat"),
  scale    = 1000,
  direction = -1,
  out_dir  = out_dir,
  prefix   = "NMR_NGA2021"
)

nmr_maps_adm1$mean_map

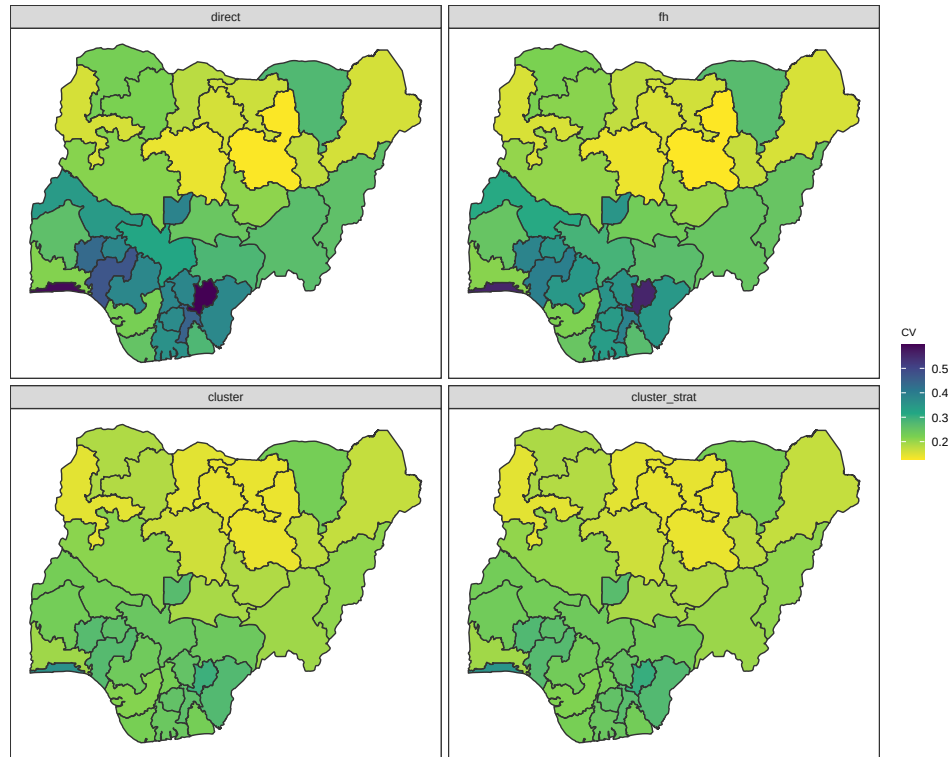
```



```
nmr_maps_adm1$interval_width_map
```



```
# Optional diagnostic: CV map
nmr_maps_adm1$cv_map
```

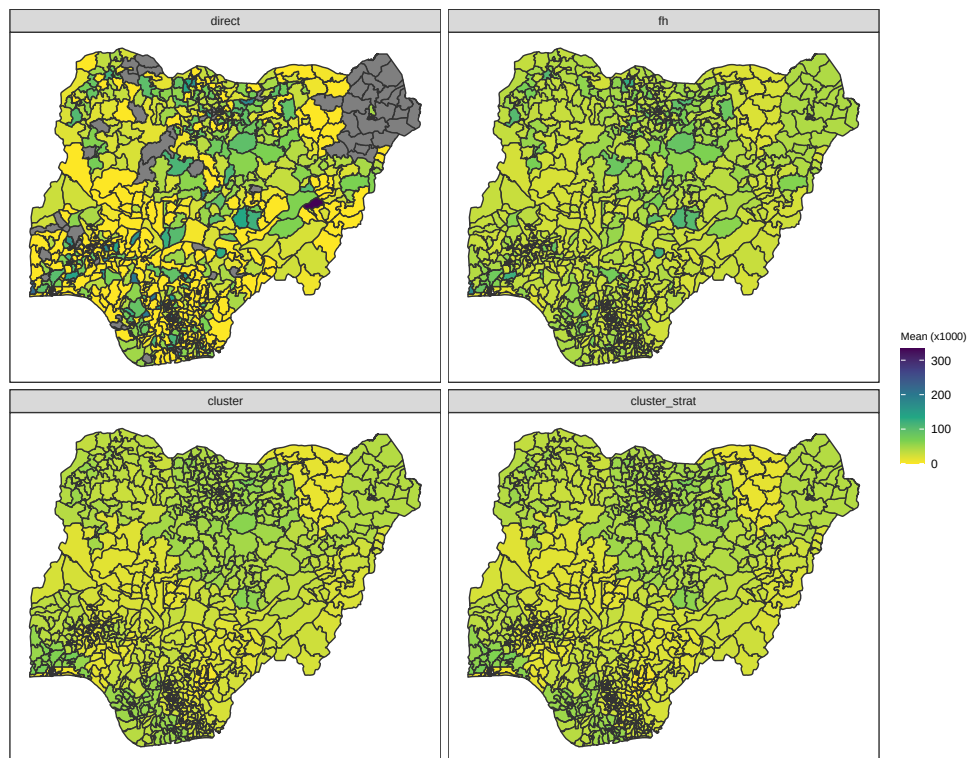


6.2 NMR maps at admin level 2

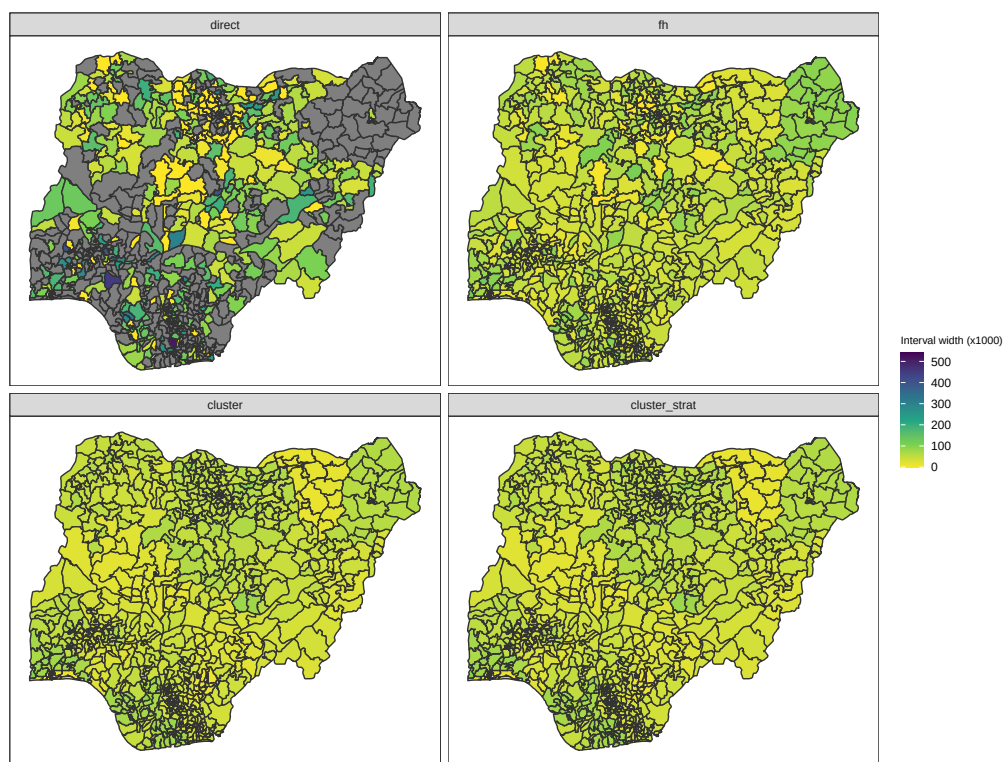
The second set of maps uses `transform = "log1p"` to improve visual contrast when most admin 2 estimates are small and a few areas have much larger values.

```
nmr_maps_adm2 <- plot_indicator_maps(
  fit = fit_nmr,
  geo = geo,
  admin = 2,
  models = c("direct", "fh", "cluster", "cluster_strat"),
  scale = 1000,
  direction = -1,
  out_dir = out_dir,
  prefix = "NMR_NGA2021"
)

nmr_maps_adm2$mean_map
```



```
nmr_maps_adm2$interval_width_map
```



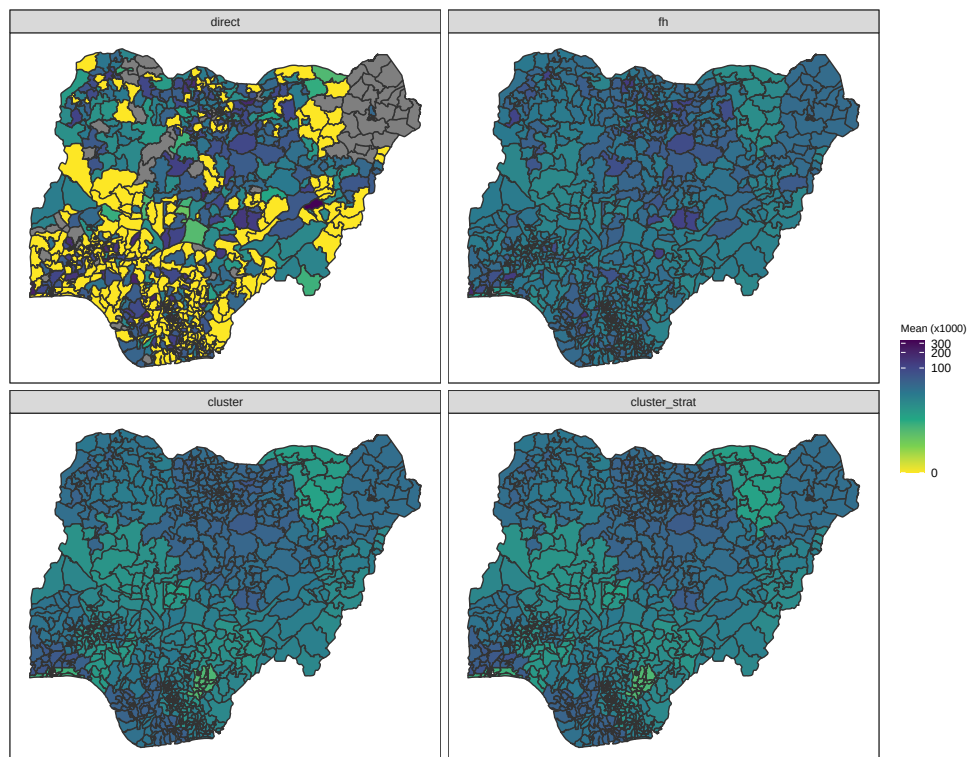
```
nmr_maps_adm2_log <- plot_indicator_maps(
```

```

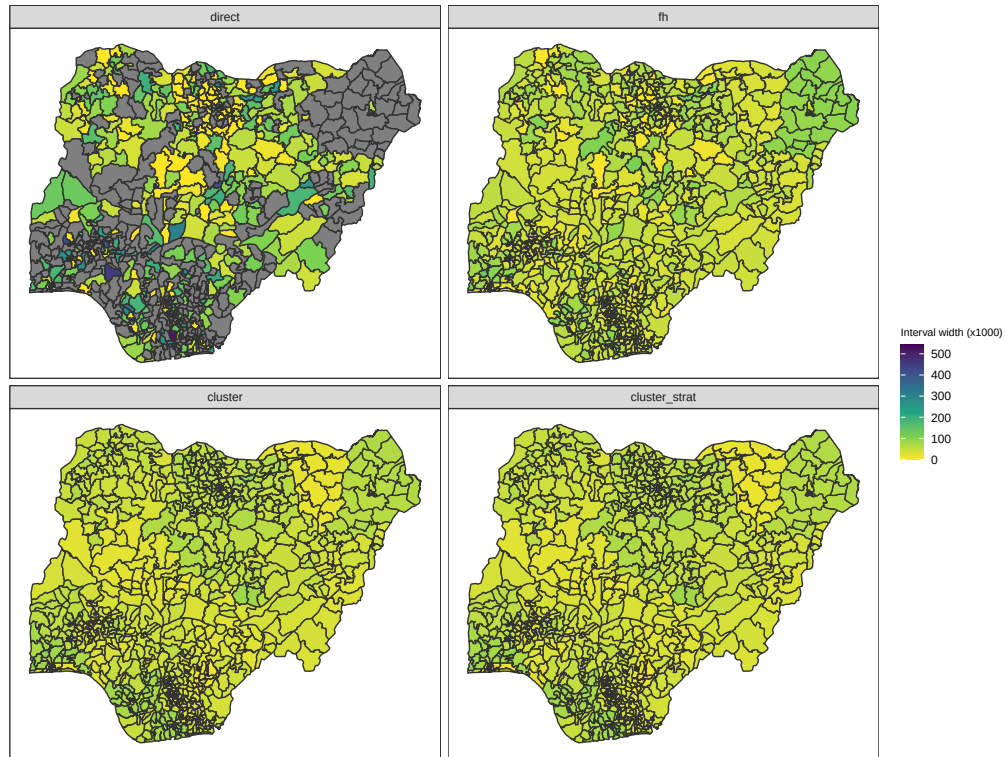
fit = fit_nmr,
geo = geo,
admin = 2,
models = c("direct", "fh", "cluster", "cluster_strat"),
scale = 1000,
direction = -1,
out_dir = out_dir,
prefix = "NMR_NGA2021_log1p",
transform = "log1p"
)

nmr_maps_adm2_log$mean_map

```

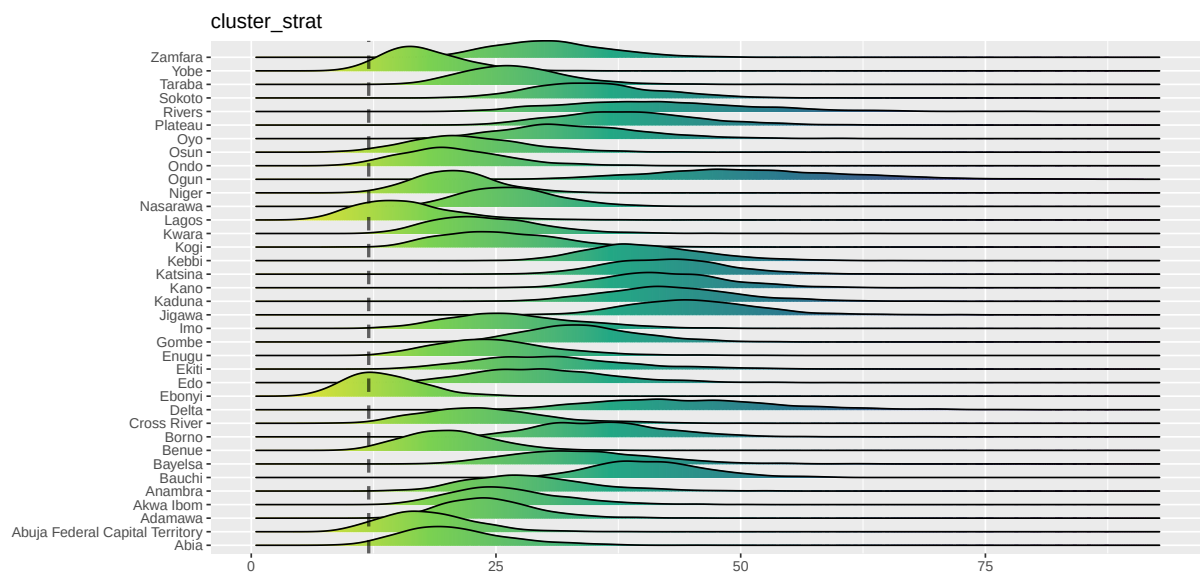
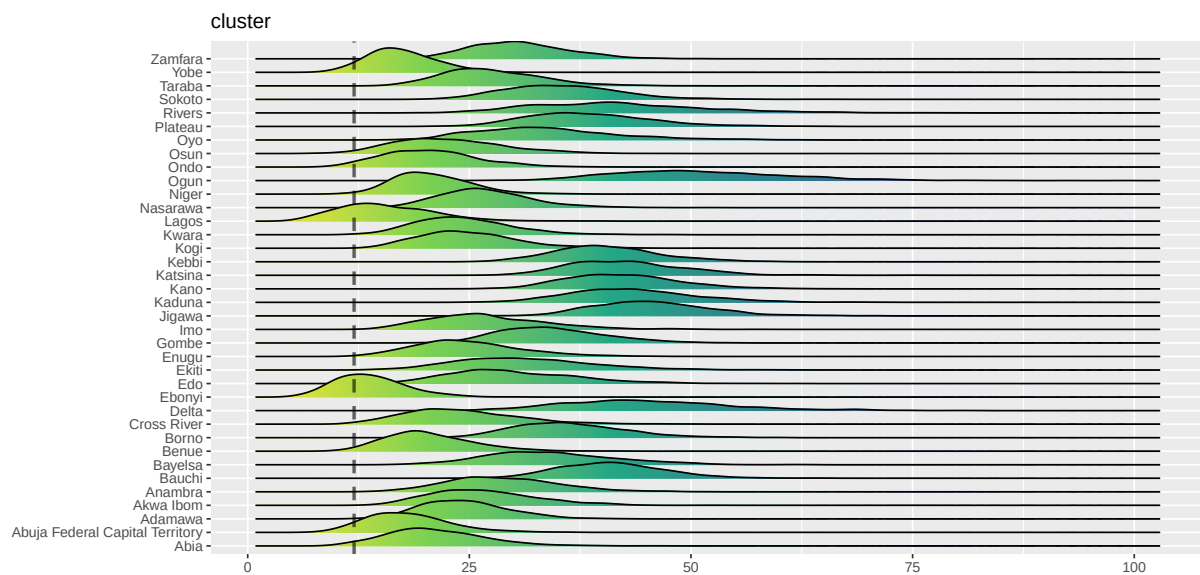
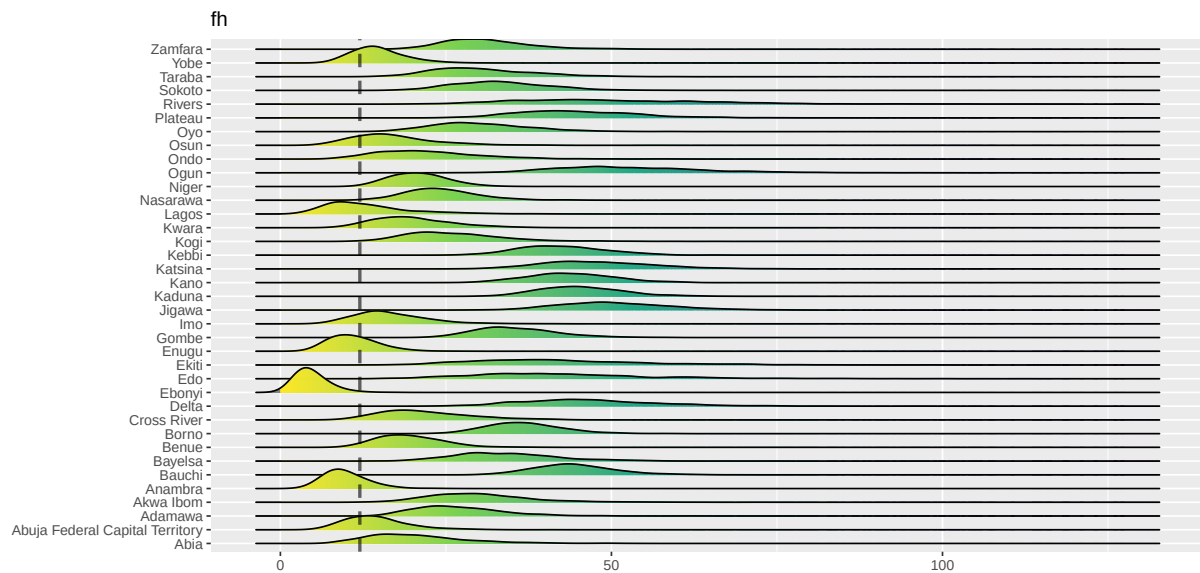


```
nmr_maps_adm2_log$interval_width_map
```



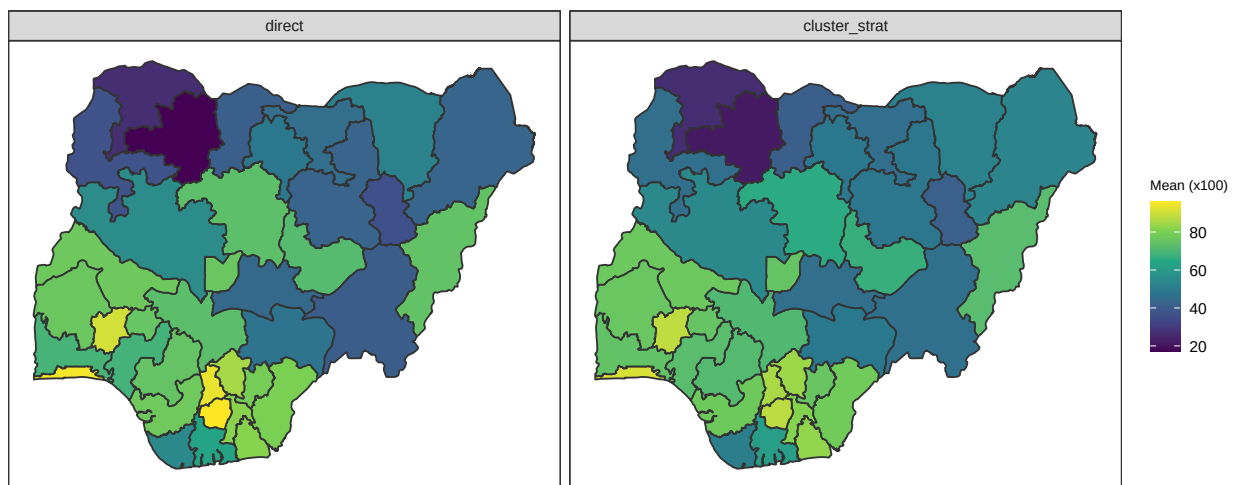
6.3 NMR ridge plots

```
p_nmr_ridge_adm1 <- plot_indicator_ridge(
  fit = fit_nmr,
  admin = 1,
  models = c("fh", "cluster", "cluster_strat"),
  threshold = 12 / 1000,
  scale = 1000,
  out_dir = out_dir,
  prefix = "NMR_NGA2021"
)
p_nmr_ridge_adm1
```

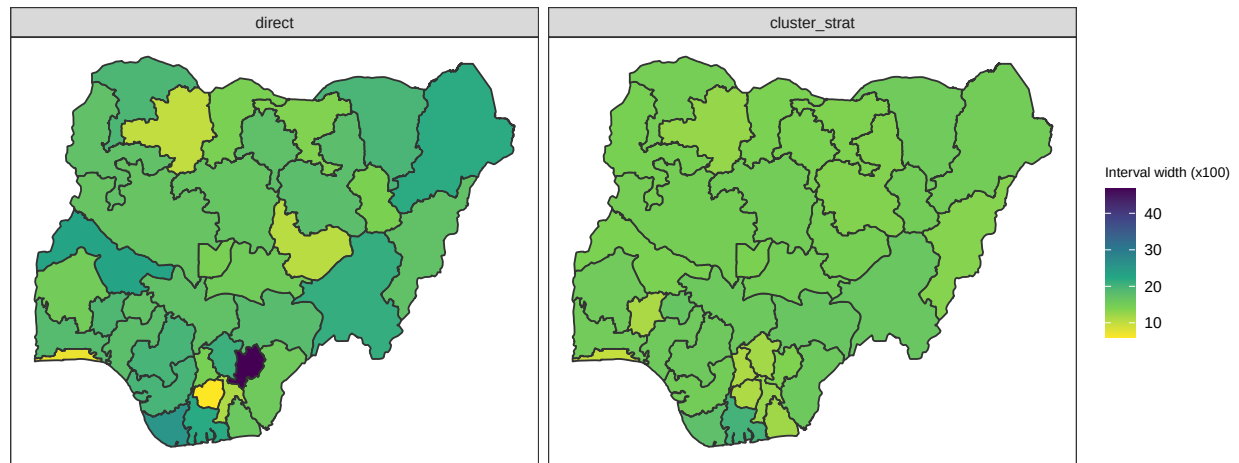


6.4 ANC maps and ridge plot

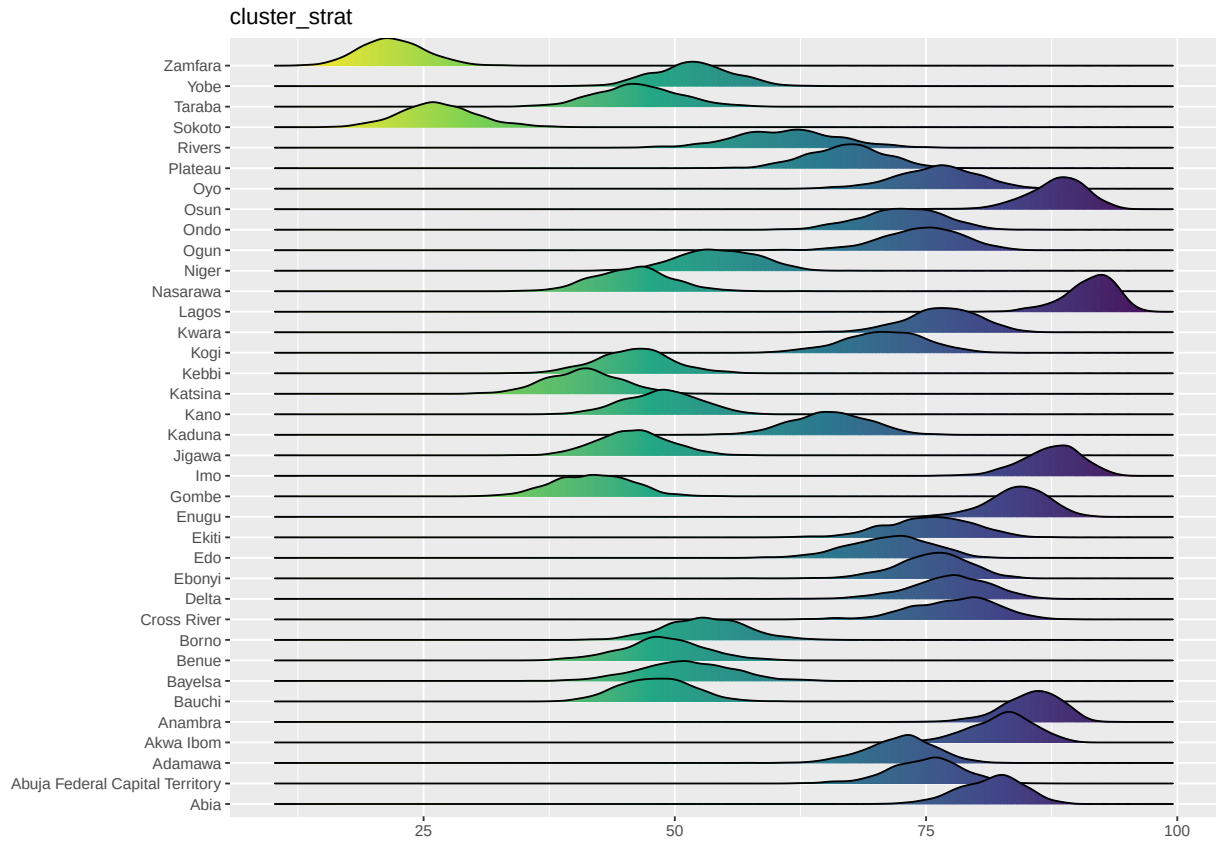
```
anc_maps_adm1 <- plot_indicator_maps(  
  fit = fit_anc,  
  geo = geo,  
  admin = 1,  
  models = c("direct", "cluster_strat"),  
  scale = 100,  
  direction = 1,  
  out_dir = out_dir,  
  prefix = "ANC4_NGA2021"  
)  
  
anc_maps_adm1$mean_map
```



```
anc_maps_adm1$interval_width_map
```



```
p_anc_ridge_adm1 <- plot_indicator_ridge(
  fit = fit_anc,
  admin = 1,
  models = c("cluster_strat"),
  threshold = NULL,
  scale = 100,
  out_dir = out_dir,
  prefix = "ANC4_NGA2021"
)
p_anc_ridge_adm1
```



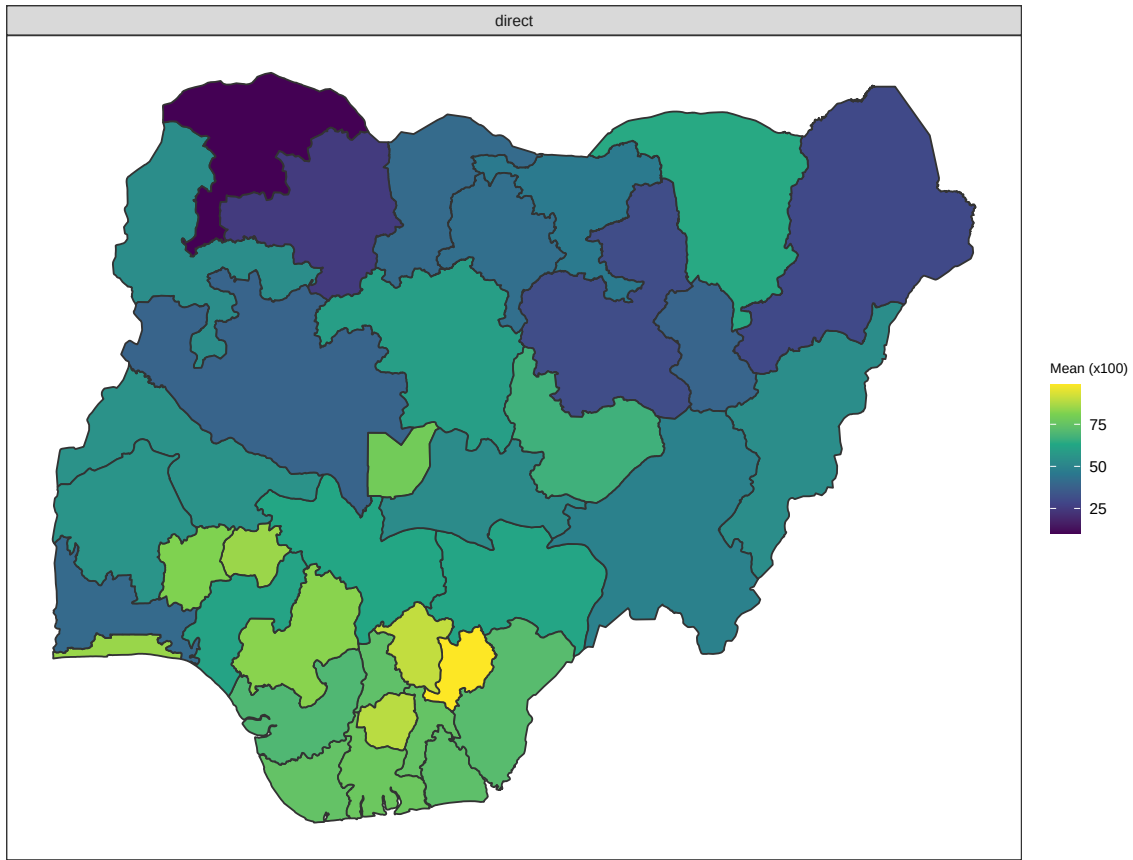
6.5 DTP3 maps

```

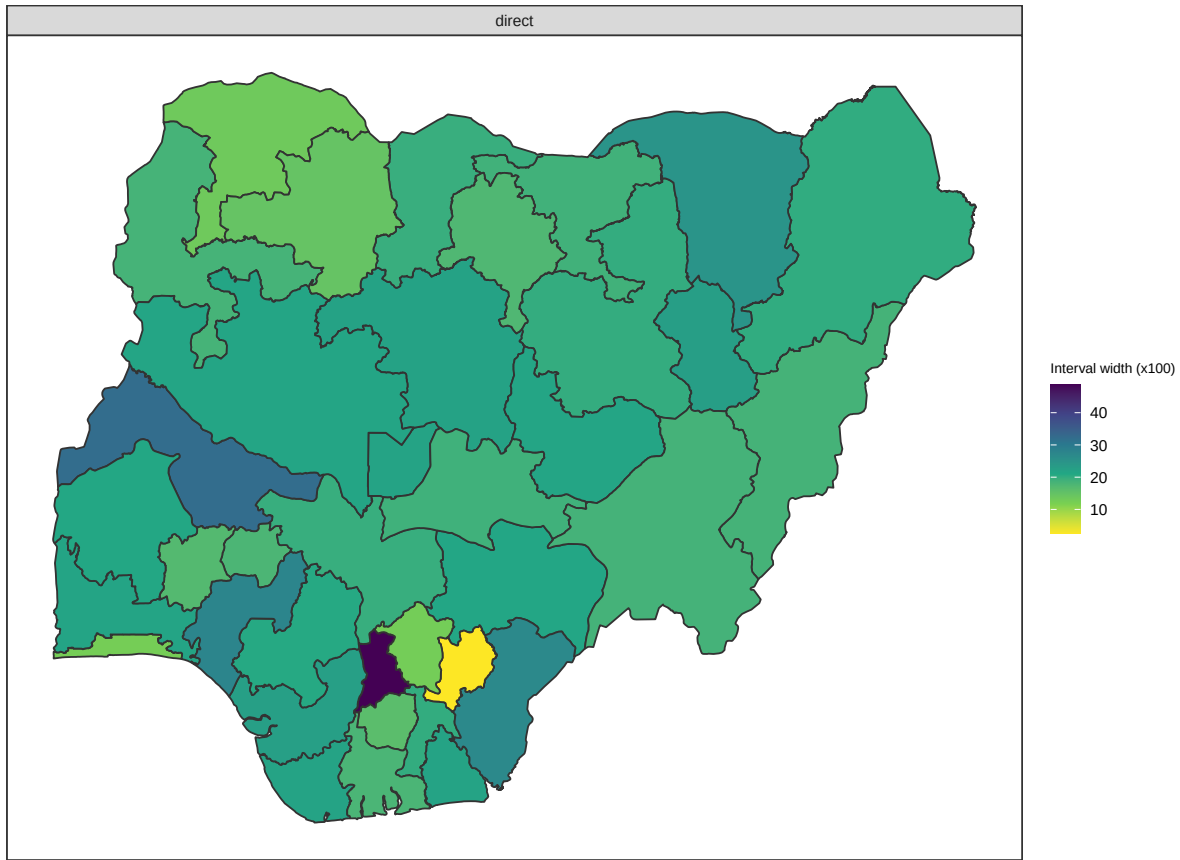
dtp3_maps_adm1 <- plot_indicator_maps(
  fit = fit_dtp3,
  geo = geo,
  admin = 1,
  models = c("direct"),
  scale = 100,
  direction = 1,
  out_dir = out_dir,
  prefix = "DTP3_NGA2021"
)

dtp3_maps_adm1$mean_map

```



```
ntp3_maps_adm1$interval_width_map
```



7. Integration with surveyPrev

MICSprev provides a streamlined wrapper (`run_indicator_models()`) to process data and run standard analyses. Advanced users may want more direct control over their modeling and visualizations.

Because MICSprev is built directly on the surveyPrev CRAN package, the datasets and spatial objects created with `process_indicator()` and `process_geo_mics()` work directly with `surveyPrev`'s functions.

If you want to perform custom model tuning, extract specific posterior samples, or use plotting options that are not in the wrapper, you can pass your MICSprev objects directly into `surveyPrev`.

7.1 Demonstration: Direct Fitting and Plotting

In this example, we use `surveyPrev` directly to fit a Fay-Herriot model for ANC coverage at Admin 1, and compare the results using its built-in `scatterPlot()` plotting function.

```
# Load the underlying modeling engine
library(surveyPrev)

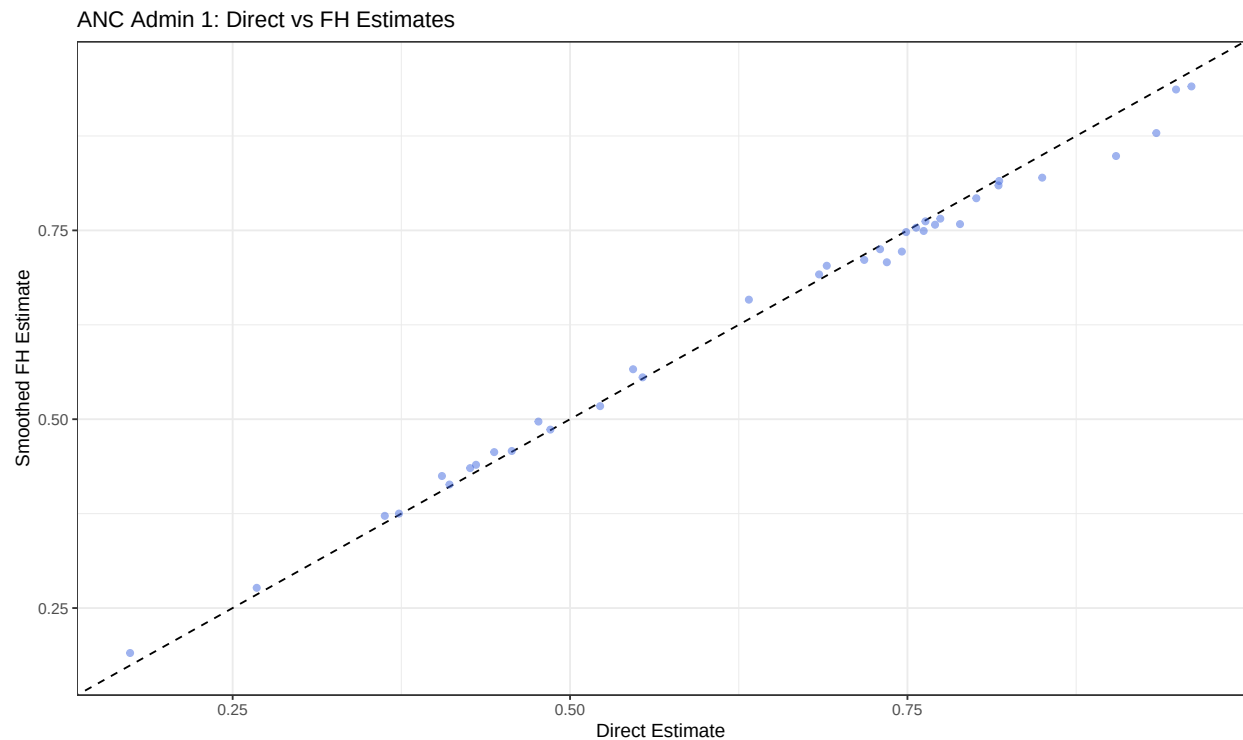
# 1. Calculate direct estimates at Admin 1
direct_res <- directEST(
  data = dat_anc,
  cluster.info = geo$cluster.info,
  admin = 1,
  aggregation = FALSE
)
```

```

# 2. Fit a Fay-Herriot (BYM2) model at Admin 1
fh_res <- fhModel(
  data = dat_anc,
  cluster.info = geo$cluster.info,
  admin.info = geo$admin.info1,
  admin = 1,
  model = "bym2",
  aggregation = FALSE
)

# 3. Use surveyPrev's built-in plotting functions
# Compare the smoothed FH estimates against the Direct estimates
scatterPlot(
  res1 = direct_res$res.admin1,
  res2 = fh_res$res.admin1,
  value1 = "direct.est",
  value2 = "mean",
  by.res1 = "admin1.name",
  by.res2 = "admin1.name",
  title = "ANC Admin 1: Direct vs FH Estimates",
  label1 = "Direct Estimate",
  label2 = "Smoothed FH Estimate"
)

```



Notes

- This vignette is written as a workflow guide using real example files shipped with the package.
- Depending on model complexity and system setup, some model-fitting or plotting steps may take time.

Session information

```
sessionInfo()
#> R version 4.5.2 (2025-10-31 ucrt)
#> Platform: x86_64-w64-mingw32/x64
#> Running under: Windows 11 x64 (build 26200)
#>
#> Matrix products: default
#> LAPACK version 3.12.1
#>
#> locale:
#> [1] LC_COLLATE=Chinese (Simplified)_China.utf8 LC_CTYPE=Chinese (Simplified)_China.utf8 LC_MONET
#> [5] LC_TIME=Chinese (Simplified)_China.utf8
#>
#> time zone: America/Los_Angeles
#> tzcode source: internal
#>
#> attached base packages:
#> [1] grid      stats      graphics  grDevices  utils      datasets  methods    base
#>
#> other attached packages:
#> [1] surveyPrev_2.0.0 survey_4.5      survival_3.8-3  Matrix_1.7-4    INLA_25.10.19  sf_1.1-1
#>
#> loaded via a namespace (and not attached):
#> [1] naniar_1.1.0           RColorBrewer_1.1-3    rstudioapi_0.18.0     jsonlite_2.0.0
#> [8] rmarkdown_2.30         ragg_1.5.2            fs_2.1.0              fields_17.3
#> [15] terra_1.9-25           SUMMER_2.0.0          htmltools_0.5.9       forcats_1.0.1
#> [22] s2_1.1.9               spData_2.3.5          KernSmooth_2.23-26    htmlwidgets_1.6.4
#> [29] lifecycle_1.0.5        pkgconfig_2.0.3       sjlabelled_1.2.0      R6_2.6.1
#> [36] patchwork_1.3.2        pkgload_1.5.2         textshaping_1.0.5     labeling_0.4.3
#> [43] remotes_2.5.0          fontquiver_0.2.1      withr_3.0.2           htmlTable_2.5.0
#> [50] DBI_1.3.0              pkgbuild_1.4.8        maps_3.4.3            MASS_7.3-65
#> [57] classInt_0.4-11        tools_4.5.2           units_1.0-1           otel_0.2.0
#> [64] callr_3.7.6            shadowtext_0.1.6      checkmate_2.3.4       generics_0.1.4
#> [71] tzdb_0.5.0            class_7.3-23          sn_2.1.3              data.table_1.18.2.1
#> [78] stringr_1.6.0          spam_2.11-3           mitools_2.4           splines_4.5.2
#> [85] fontLiberation_0.1.0   knitr_1.51            fontBitstreamVera_0.1.1 gridExtra_2.3
#> [92] matrixStats_1.5.0      stringi_1.8.7         yaml_2.3.12           boot_1.3-32
#> [99] tibble_3.3.1          cli_3.6.5            systemfonts_1.3.2     processx_3.9.0
#> [106] ellipsis_0.3.3         fmesher_0.7.0        storr_1.2.6           ggplot2_4.0.2
#> [113] ggiraph_0.9.6          scales_1.4.0         e1071_1.7-17          ggridges_0.5.7
#> [120] rlang_1.2.0           mnormt_2.1.2
```